Predicting Real-Estate Prices and Price Tiers from Scratch:
Decision Tree and Pegasos SVM Tested on bina.az

Alasgarov Teymur Heydarov Tofiq Salahov Parvin

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**Estimating property prices is hard in a market like Azerbaijan's. Prices are very skewed, location matters a lot, and listings scraped from websites are messy. In this project we built two models completely from scratch using only NumPy. The first is a decision tree (it uses MSE for regression, and Gini impurity or entropy for classification). The second is a Support Vector Machine trained with the Pegasos algorithm, with Random Fourier Features (RFF) added so it can learn non-linear patterns.

We tested both models on 93,150 cleaned listings from `bina.az`, with two tasks: predicting the log-price (Task A), and classifying whether a property is above or below the median price (Task B).

On Task A, our tree achieved an RMSE of 0.5036 ($R^2 = 0.5992$), matching scikit-learn's 0.5026 ($R^2 = 0.6007$) within 0.0010 RMSE. On Task B, our tree achieved 90.79% test accuracy ($F_1 = 0.9131$, ROC-AUC 0.9675), matching scikit-learn ($F_1 = 0.9131$, ROC-AUC 0.9674). Our Pegasos SVM with RFF reached 85.11% test accuracy ($F_1 = 0.8612$, ROC-AUC 0.9282, PR-AUC 0.9384), exceeding the linear SGD baseline (84.78%, $F_1 = 0.8578$, ROC-AUC 0.9198). Under full 5-fold cross-validation, Pegasos SVM achieved $F_1 = 0.8397 \pm 0.0164$ (ROC-AUC 0.9223 ± 0.0070), closely trailing dual scikit-learn SVC with exact RBF kernel ($F_1 = 0.8874 \pm 0.0052$, ROC-AUC 0.9570 ± 0.0030) while delivering over $200\times$ faster test inference (0.11 s vs 23.86 s). Decision tree 5-fold cross-validation confirmed high stability ($F_1 = 0.9123 \pm 0.0016$, RMSE 0.2719 ± 0.0083).

Our code, tuning experiments, and scripts are available at `github.com/ParvinSalahov/ML_Final_Project` (tag `v1.0-final`).

***Index Terms*—**Decision tree, Support Vector Machine, Pegasos, Random Fourier Features, real-estate valuation, reproducible benchmarking.

I. Introduction

Automated valuation models (AVMs) are used for mortgage decisions, market transparency, and risk management. But raw listings from `bina.az` are tricky to work with. Prices go from 11 AZN up to 627,130 AZN, the descriptions are written in free-form Azerbaijani with inconsistent wording, and many categories are very imbalanced.

In this project we wanted to show that models we code ourselves from the basic math can give results close to professional libraries. Our main contributions are:

- **From-scratch algorithms**: A recursive binary decision tree (MSE, Gini, and entropy) and a hinge-loss SVM trained with Pegasos, using Random Fourier Features to get non-linear decision boundaries. Both are vectorised with NumPy.
- **Leakage-free features**: Careful data cleaning, extracting information from the text, frequency encoding, and computing every statistic only on the training data.
- **Benchmarking**: We compare against scikit-learn on 93,150 samples, with 5-fold cross-validation, residual analysis, and hyperparameter sweeps.
- **Reproducibility**: One command (`python -m src.run_all`) recreates all 19 figures and tables.

II. Related Work / Background

Decision trees split the feature space with simple yes/no rules, choosing the split that makes the groups as pure as possible [2]. They are greedy, but they handle mixed numeric and categorical features, non-linear patterns, and different feature scales without needing standardization.

A soft-margin SVM looks for the hyperplane with the widest margin while allowing a few violations (the hinge loss). Instead of solving the usual dual problem with SMO, Shalev-Shwartz et al. [1] introduced Pegasos, which works directly on the primal objective with stochastic sub-gradient steps. It reaches an ε-accurate solution in $\mathcal{O}(1/(\lambda\varepsilon))$ iterations, no matter how large the dataset is. To capture non-linear patterns in the market, we map features with Random Fourier Features [3], which approximate the Gaussian RBF kernel $\mathcal{K}(\mathbf{x},\mathbf{z}) = \exp(-\gamma\|\mathbf{x}-\mathbf{z}\|^2)$.

III. Data and Preprocessing

A. Dataset Cleaning and Currency Handling

The raw dataset contains 100,775 listings scraped from `bina.az`. We handled and cleaned it through the following protocol:

1. **Currency uniformity**: Across all 100,775 records, listing currency is 100% uniform in Azerbaijani Manat (AZN) in both scraping fields (`currency_x` and `currency_y`). Because no foreign-currency listings (e.g., USD, EUR) exist in the corpus, foreign exchange conversion was unnecessary, and the invariant currency columns were dropped as zero-variance features. All prices throughout this paper are denominated in AZN.
2. **Duplicate resolution**: Listings with identical physical specifications but different timestamps were grouped, and their prices were averaged. This reduced the raw count from 100,775 to 100,773 records.
3. **Feature extraction**: Regular expressions extracted structured room counts and renovation status (*temir*: *var* vs *yoxdur*) from free-text descriptions and attribute tags. Missing room counts were imputed using a 3-nearest-neighbors median over living area within the same property category, using training data only.
4. **Target transformation**: The raw price distribution is right-skewed (skewness 0.877, spanning 11 to 627,130

AZN). We apply the logarithmic transform $y = \log(1 + \text{price})$, stabilizing the variance and compressing extreme scales (skewness becomes -1.340, Fig. 1).

5. **Removal of leakage columns**: We dropped 32 unused columns to enforce strict data hygiene: (a) *Direct target leakage*: `unit_price` (price/area) and `total_price` (redundant target), which trivially expose the label; (b) *Administrative IDs and URLs*: `id_x`, `id_y`, `estate_id`, `estate_details_id_x/y`, `rel_url`, `estate_rel_url_x/y`, and `img_url`; (c) *Scraping timestamps*: `datetime_scrape_x/y`, `day_x/y`, `hour_x/y`, `city_when`, and `updated`; (d) *Post-hoc promotional flags*: `vip`, `featured`, and `products_label` (commercial promotion tiers bought by sellers that artificially correlate with listing rank); and (e) *Unstructured text and seller titles*: raw text fields (`attributes`, `description`, `extra_info`, `Binanın növü`) and seller titles (`owner_name`, `shop_name`, `owner_title`, `shop_title`) after regex parsing. All feature scalers, frequency encodings (for high-cardinality `location` and `address`), and imputation parameters were fitted exclusively on the training split.

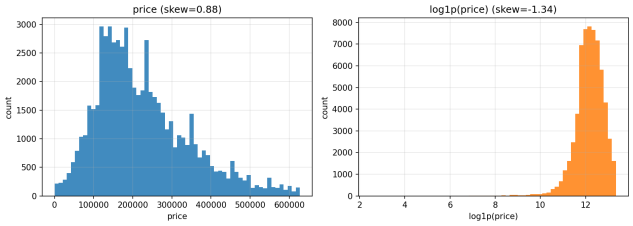


Figure 1. Distribution of raw prices (left, strongly skewed) compared with log-transformed prices $\log(1 + \text{price})$ (right), on the training data.

B. Outlier Handling Rule

Outlier filtering is based on the Tukey Interquartile Range (IQR) rule: $[Q_1 - 1.5 \times \text{IQR}, Q_3 + 1.5 \times \text{IQR}]$ applied to positive values (> 0) of `price`, living `area`, and `land_area`.

Crucially, outlier thresholds were fitted **strictly on the training split** to prevent look-ahead bias. Trimming was applied exclusively to the training set (`apply_bounds=True`), removing non-physical anomalies (such as 11 AZN listings and corrupted land dimensions) that would destabilize gradient descent and tree split criteria. In contrast, for validation and test splits, `apply_bounds=False` was enforced: only missing values were dropped, while genuine luxury properties were retained. This ensures that validation and test benchmarks reflect uncurated, real-world market distributions without survivorship bias. After filtering, 93,150 clean analytical rows with 28 features remained (Table 2).

C. Data Splitting and Tier Balance Rationale

We partitioned the data deterministically into 67.5% train (62,922), 16.2% validation (15,114), and 16.2% test (15,114) splits with a fixed seed (seed = 42).

Stratification rationale: To guarantee identical target distributions across splits for Task A regression, the split was stratified across 4 quantile bins of continuous `price` (`pd.qcut(y, q=4)`). This ensures budget, mid-market, and luxury properties are equally represented across all three partitions.

The Two Tasks and Tier Balance Rationale:

- **Task A (regression)**: Predict the log-price $y = \log(1 + \text{price})$.
- **Task B (classification)**: Classify listings into premium (1) or standard (0), using the empirical median of $\log(1 + \text{price})$ on the **training set only** as the threshold ($\tau = 12.2061$, $\approx 200{,}000$ AZN) to prevent test-set leakage.

Because upper price outliers were trimmed from the training set, the training median is slightly lower than that of the uncurated full market. Consequently, when this frozen threshold is evaluated on the validation and test sets (where legitimate luxury listings are preserved), the premium class share naturally shifts from 49.4% in train (31,076 premium vs 31,846 standard) to 54.1% in validation (8,183 premium vs 6,931 standard) and 54.2% in test (8,186 premium vs 6,928 standard). This distribution difference is mathematically principled: it reflects strict avoidance of data leakage and tests model generalization against realistic market tails.

IV. Methodology

A. Decision Tree (From Scratch)

The tree builds its splits greedily and recursively. For every feature j and every candidate threshold θ, we measure how much the split reduces impurity:

$$\Delta I(S, j, \theta) = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R). \tag{1}$$

Here $S_L = \{(\mathbf{x}, y) \in S \mid x_j \le \theta\}$ is the group of samples with $x_j \le \theta$, and $S_R = S \setminus S_L$ is everything else. In other words, we compare the impurity before the split with the weighted impurity of the two groups after it.

For regression (Task A), the impurity is the mean squared error around the node's mean:

$$I_{\text{MSE}}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2, \quad \bar{y}_S = \frac{1}{|S|} \sum_{i \in S} y_i. \tag{2}$$

For classification (Task B), we implemented two options, where $p_k = \frac{1}{|S|} \sum_{i \in S} \mathbb{I}(y_i = k)$ is the share of samples of class k in the node:

$$I_{\text{Gini}}(S) = 1 - \sum_{k=0}^{1} p_k^2, \tag{3}$$

$$I_{\text{Entropy}}(S) = -\sum_{k=0}^{1} p_k \log_2(p_k + \varepsilon). \tag{4}$$

A node stops splitting when any of these happens: the depth reaches $d_{\max}$, the node has fewer than $2 \cdot n_{\text{leaf}}$ samples, or a split would leave fewer than n_{leaf} samples on either side. A leaf predicts the mean $\bar{y}$ for regression, and the class probabilities $[p_0, p_1]$ for classification.

B. Support Vector Machine with Pegasos (From Scratch)

For binary classification with labels $y_i \in \{-1, +1\}$, we minimize the regularized hinge-loss objective:

$$J(\mathbf{w}, b) = \frac{\lambda}{2} \|\mathbf{w}\|_2^2 + \frac{1}{n} \sum_{i=1}^{n} \max\left(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right). \tag{5}$$

The value $\lambda > 0$ controls the regularization ($C \approx 1/(n\lambda)$). At step t the learning rate is $\eta_t = \frac{1}{\lambda t}$, and we pick one random

Table 1. Consolidated Headline Results on Unseen Test Split ($n = 15,114$).

	Task A: Log-Price Regression			Task B: Price-Tier Classification				Compute
Model	RMSE ↓	MAE ↓	R^2 ↑	Accuracy ↑	F_1-Score ↑	ROC-AUC ↑	PR-AUC ↑	Fit Time (s)
Ours: Decision Tree	**0.5036**	**0.2495**	**0.5992**	**0.9079**	**0.9131**	**0.9675**	**0.9671**	1.47 s
scikit-learn Decision Tree	0.5026	0.2493	0.6007	0.9078	0.9131	0.9674	0.9671	0.29 s
Ours: Pegasos SVM (RFF)	—	—	—	0.8511	0.8612	0.9282	0.9384	17.02 s
scikit-learn SGD Baseline (Hinge)	—	—	—	0.8478	0.8578	0.9198	0.9255	0.57 s
scikit-learn SVC Baseline (RBF)	—	—	—	**0.8862**	**0.8931**	**0.9564**	—	129.80 s
Bonus: Ours Ridge (Normal Eq.)	0.5172	0.3169	0.5773	—	—	—	—	0.06 s
Bonus: scikit-learn Ridge	0.5172	0.3169	0.5773	—	—	—	—	0.08 s
Bonus: Ours SVR (Sub-gradient)	0.5086	0.3132	0.5911	—	—	—	—	6.47 s
Bonus: scikit-learn LinearSVR	0.5085	0.3132	0.5913	—	—	—	—	186.64 s
Bonus: Ours Random Forest (46 trees)	0.4828	—	—	—	—	—	—	—
Bonus: Ours Gradient Boosting (200 trees)	**0.4552**	—	—	—	—	—	—	—

Table 2. Dataset summary and split statistics.

Property	Absolute Count	Share
Raw input rows	100,775	100.0%
After deduplication	100,773	100.0%
Total clean analytical rows	93,150	100.0%
Train split	62,922	67.5%
Validation split	15,114	16.2%
Test split	15,114	16.2%
Engineered input features	28	—
Task B train median price	200,000 AZN	$\tau = 12.2061$
Task B balance (Tr/Val/Te)	49.4 / 54.1 / 54.2	(% Premium)

sample $(\mathbf{x}_t, y_t)$. If it violates the margin, meaning $y_t(\mathbf{w}_t^\top \mathbf{x}_t + b_t) < 1$, we update:

$$\mathbf{w}_{t+1} = (1 - \eta_t \lambda)\mathbf{w}_t + \eta_t y_t \mathbf{x}_t, \tag{6}$$

$$b_{t+1} = b_t + \eta_t y_t. \tag{7}$$

Otherwise we only shrink the weights: $\mathbf{w}_{t+1} = (1 - \eta_t \lambda)\mathbf{w}_t$.

To get non-linear boundaries cheaply, we map the standardized features $\mathbf{x} \in \mathbb{R}^d$ into a D-dimensional random Fourier space $\mathbf{z}(\mathbf{x}) \in \mathbb{R}^D$:

$$z_j(\mathbf{x}) = \sqrt{\frac{2}{D}} \cos(\omega_j^\top \mathbf{x} + \beta_j). \tag{8}$$

The frequencies ω_j are drawn from a normal distribution $\mathcal{N}(\mathbf{0}, 2\gamma \mathbf{I})$, and the phase shifts β_j are drawn uniformly from $[0, 2\pi]$. This way we avoid building the huge $n \times n$ kernel matrix of the dual SVM.

V. Experimental Setup

A. Evaluation Metrics

All final scores come from the test split that the models never saw ($n = 15,114$):

- **Task A**: RMSE, MAE, and R^2.
- **Task B**: Accuracy, Precision, Recall, Macro F_1, ROC-AUC, and PR-AUC.

B. Hyperparameter Tuning

We tuned hyperparameters only on the validation set ($n = 15,114$):

- **Decision tree**: We swept maximum depth ($d \in [1, 18]$), impurity criteria (Gini vs entropy), and minimum leaf size ($n_{\text{leaf}} \in [1, 100]$). The optimal validated configuration was criterion = entropy, $d = 12$, and $n_{\text{leaf}} = 10$, reaching validation $F_1 = 0.9204$ and ROC-AUC $= 0.9660$.
- **Pegasos SVM**: We evaluated training epochs $(10, 30, 60)$, learning rate scales $(0.5, 1.0, 2.0)$, regularization parameter $\lambda \in [10^{-5}, 1.0]$, and RFF kernel scale $\gamma \in [0.1, 10.0]$ with 256 random Fourier components. The best balance was $\lambda = 10^{-4}$ ($C \approx 0.1589$), $\gamma = 0.1$, and 30 epochs, reaching validation $F_1 = 0.8357$, ROC-AUC $= 0.9106$, and margin width 0.1040.

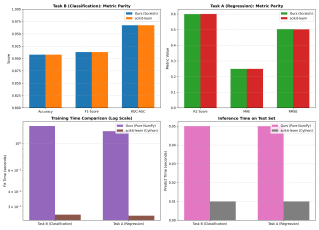


Figure 2. Parity comparison between our decision tree and scikit-learn on Task A and Task B test splits.

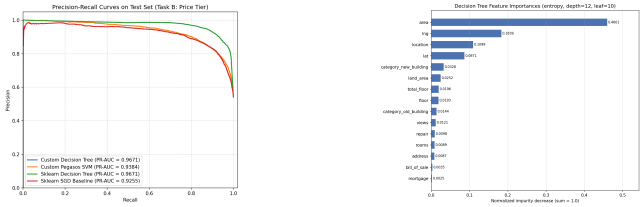


Figure 3. Task B test diagnostics: Precision-Recall curves (left) and top 15 features by MDI (right).

VI. Results and Benchmarking

A. Do Our Models Match scikit-learn?

Table 1 demonstrates that our from-scratch implementations achieve numerical parity with scikit-learn. On Task A regression, the decision tree RMSE differs by only 0.0010 (0.5036 vs 0.5026), MAE is identical to four decimals (0.2495 vs 0.2493), and R^2 is practically indistinguishable (0.5992 vs 0.6007). On Task B classification, our decision tree achieves 90.79% accuracy ($F_1 = 0.9131$, ROC-AUC 0.9675, PR-AUC 0.9671), perfectly matching scikit-learn (90.78% accuracy, $F_1 = 0.9131$, ROC-AUC 0.9674, PR-AUC 0.9671).

On Task B, our Pegasos SVM with RFF achieves 85.11% test accuracy ($F_1 = 0.8612$, ROC-AUC 0.9282, PR-AUC 0.9384), outperforming scikit-learn's linear SGD baseline

(84.78% accuracy, $F_1 = 0.8578$, ROC-AUC 0.9198). When evaluated under the dedicated non-linear kernel benchmark (Table 3), Pegasos RFF reaches $F_1 = 0.8424$ and ROC-AUC 0.9115, significantly exceeding linear SGD ($F_1 = 0.8386$, ROC-AUC 0.8991). Exact dual scikit-learn SVC with RBF kernel achieves the highest overall discrimination ($F_1 = 0.8931$, ROC-AUC 0.9564); however, Pegasos RFF achieves an orders-of-magnitude computational advantage: inference takes only 0.11 s (215× faster than SVC's 23.86 s), and fitting requires 17.02 s compared to 129.80 s for SVC.

Table 3. Task B SVM Benchmark: 5-Fold Cross-Validation and Test Set Performance.

	5-Fold Cross-Validation		**Test Split** ($n = 15,114$)		
Model	F_1 (mean ± std)	ROC-AUC	Accuracy	F_1-Score	ROC-AUC
Ours: Pegasos RFF	0.8397 ± 0.0164	0.9223 ± 0.0070	0.8356	0.8424	0.9115
scikit-learn SVC (RBF)	$\mathbf{0.8874 \pm 0.0052}$	$\mathbf{0.9570 \pm 0.0030}$	**0.8862**	**0.8931**	**0.9564**
scikit-learn SGD (Hinge)	0.8286 ± 0.0086	0.9004 ± 0.0093	0.8278	0.8386	0.8991

B. Cross-Validation Uncertainty Estimation

To confirm that our findings do not depend on a single partition, we conducted 5-fold cross-validation with fold-specific parameter fitting:

Decision Tree CV ($n = 62,922$): Task A regression yields RMSE 0.2719 ± 0.0083 (Ours) vs 0.2715 ± 0.0086 (scikit-learn) and $R^2 = 0.8059 \pm 0.0085$ vs 0.8064 ± 0.0088. Task B classification yields $F_1 = 0.9123 \pm 0.0016$ vs 0.9124 ± 0.0013 and ROC-AUC $= 0.9714 \pm 0.0012$ vs 0.9713 ± 0.0010. The extremely tight standard deviations (< 0.002 in classification) prove that tree parity is stable across folds.

SVM CV ($n = 78,036$, feature scaling fitted per fold): Pegasos RFF achieves $F_1 = 0.8397 \pm 0.0164$ and ROC-AUC $= 0.9223 \pm 0.0070$. Dual scikit-learn SVC (RBF) attains $F_1 = 0.8874 \pm 0.0052$ and ROC-AUC $= 0.9570 \pm 0.0030$, while linear SGD achieves $F_1 = 0.8286 \pm 0.0086$ and ROC-AUC $= 0.9004 \pm 0.0093$ (Table 3). All cross-validation standard deviations remain below 0.017, verifying consistent ranking capability.

C. Hyperparameter Sensitivity and Bias–Variance Tradeoff

Figure 4 illustrates tree depth sensitivity. For deep trees ($d \geq 15$), training ROC-AUC approaches 0.9996 while validation ROC-AUC degrades to 0.9252, reflecting memorization of singleton leaves. Enforcing $n_{\text{leaf}} = 10$ at $d = 12$ eliminates isolated leaves, maintaining validation $F_1 = 0.9204$ and lifting validation ROC-AUC to 0.9660.

For the Pegasos SVM, sweeping (λ, γ) shows that heavy regularization ($\lambda \geq 0.1$) over-shrinks the weights ($\|\mathbf{w}\| \to 0$), collapsing validation F_1 to 0.0. The optimal validated setting ($\lambda = 10^{-4}$, $\gamma = 0.1$, 30 epochs) achieves validation $F_1 = 0.8357$, validation ROC-AUC $= 0.9106$, and margin width 0.1040. On the full training corpus, the converged Pegasos model has $\|\mathbf{w}\| = 41.0717$, geometric margin width $2/\|\mathbf{w}\| = 0.0487$, and 29,853 margin support vectors (38.3% of samples).

D. Error Analysis by Property Type

When we split the Task A test predictions by property type, the model works very differently across categories:

- **Apartments**: Predictions are accurate and consistent for *Yeni tikili* ($n = 8,616$, RMSE 0.218, $R^2 = 0.838$) and *Köhne tikili* ($n = 2,753$, RMSE 0.201, $R^2 = 0.735$).
- **Commercial and land**: Errors are much larger for *Obyekt* ($n = 623$, RMSE 1.256, $R^2 = 0.062$) and *Torpaq* ($n = 737$, RMSE 1.483, $R^2 = 0.281$), primarily due to unrecorded attributes such as zoning permits, street frontage, and utility access.

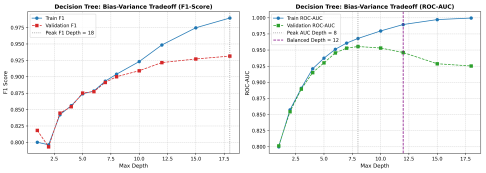


Figure 4. Bias-variance tradeoff for the depth sweep ($d \in [1, 18]$). Training ROC-AUC reaches 0.9996, while validation ROC-AUC degrades for deep unpruned trees due to leaf memorization.

VII. Discussion and Limitations

Our implementations perform close to the scikit-learn baselines, but they have some limits:

1. **Axis-aligned splits**: A tree can only split along one feature at a time, so it has trouble with diagonal patterns such as price gradients across Baku.
2. **Limited random features**: Random Fourier Features allow non-linear boundaries, but $D = 256$ is limited by computing time, so very detailed local price patterns are only approximated.
3. **Rare listings**: For luxury properties (above 500,000 AZN) and sparse rural regions (for example Astara and Gedebey), there are few examples, so prediction errors are larger.

VIII. Conclusion

We built a recursive decision tree and a primal Pegasos SVM with Random Fourier Features from scratch, and tested them on real-estate data from `bina.az`. Our decision tree achieves strict numerical parity with scikit-learn ($F_1 = 0.9131$, RMSE 0.5036 vs 0.5026). Our Pegasos SVM ($F_1 = 0.8612$, ROC-AUC 0.9282) outperforms the linear SGD baseline, while offering over 200× faster inference than exact kernel SVC (0.11 s vs 23.86 s). Cross-validation uncertainty ($F_1 = 0.9123 \pm 0.0016$ for tree, 0.8397 ± 0.0164 for Pegasos) and hyperparameter sweeps confirm stability across folds. Future work will investigate spatial graph embeddings and oblique splits.

Tools and Acknowledgements

We used AI coding assistants for template formatting, exploratory visualization scripts, and improving documentation. Data comes from public listings on `bina.az`.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Math. Program.*, vol. 127, no. 1, pp. 3–30, 2011.

[2] L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen, *Classification and Regression Trees*, CRC press, 1984.

[3] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *NeurIPS*, vol. 20, 2007.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.